# MuGenDPO

DeepMIR Final Project

**R12942148 電信丙 林峻佑**

**R12942143 電信丙 林翰莘**

# Outline


01 Motivation

02 Methodology

03 Experiment Results

04 Conclusion

05 References

01

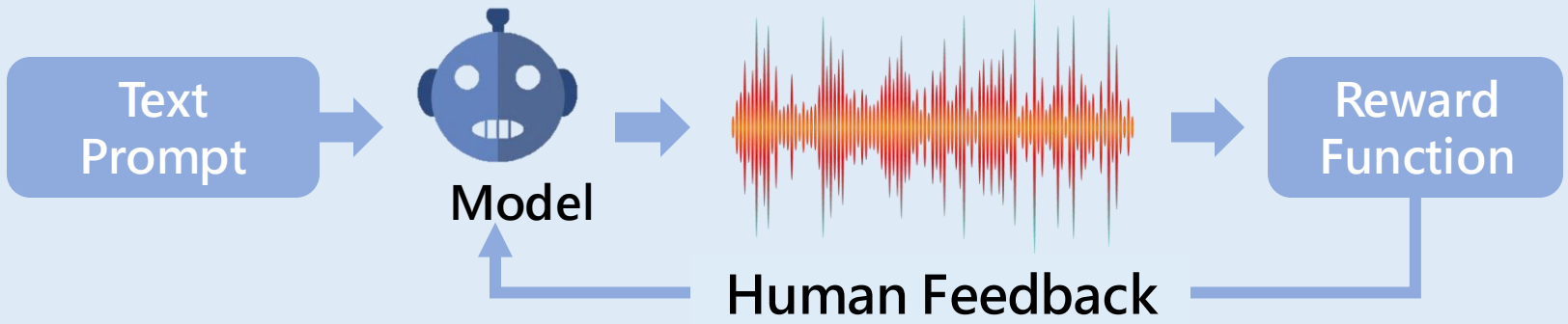
# Motivation

# Motivation

## Our Goal

**Build a text-to-music generation model to generate classical music**

## High Level Methodology




## Why RLHF (Reinforcement Learning with Human Feedback)?

- Music generation is hard to adopt objective metrics to verify the performance
- The quality of synthesized music mostly relies on the audience feedback, and it should be aligned with human preference.

02

# Methodology

# Methodology

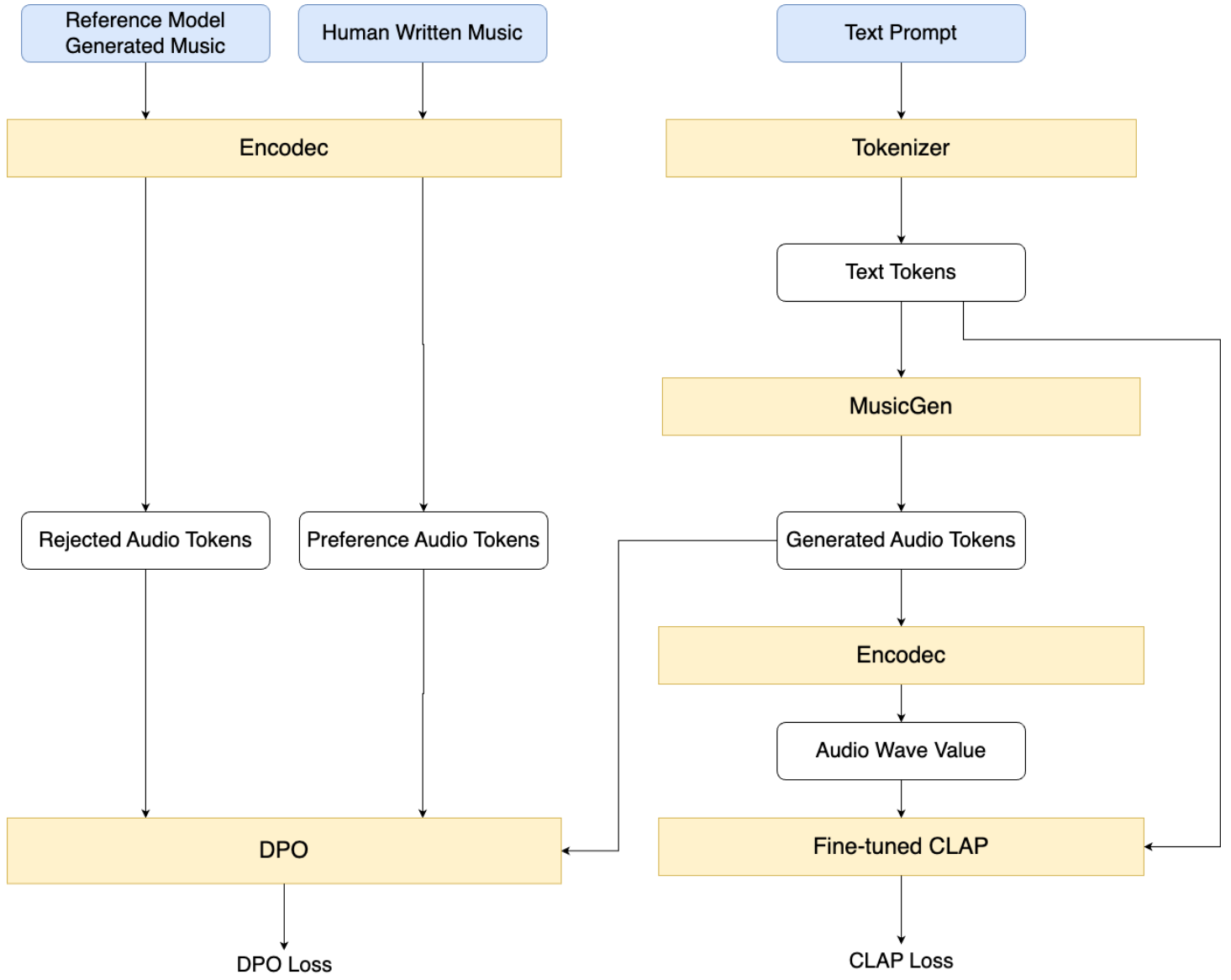
## Dataset – MusicNet

- 330 Classical Music Recordings
- Duration: 50 secs to 7 mins
- Composers: From Baroque to Impressionist, Bach and Beethoven account for a significant proportion
- Preprocessing
  - Text prompt - Classical music pairing data
  - Audio length: 25s chunk from various composers, instruments and keys
  - Dataset size: 1300 chunks
  - Dataset Example

```
id,prompt
1727,"Romantic, Piano Quintet, A major"
1728,"Romantic, Piano Quintet, A major"
1729,"Romantic, Piano Quintet, A major"
1730,"Romantic, Piano Quintet, A major"
1733,"Romantic, Solo Piano, A major"
1734,"Romantic, Solo Piano, A major"
1735,"Romantic, Solo Piano, A major"
```

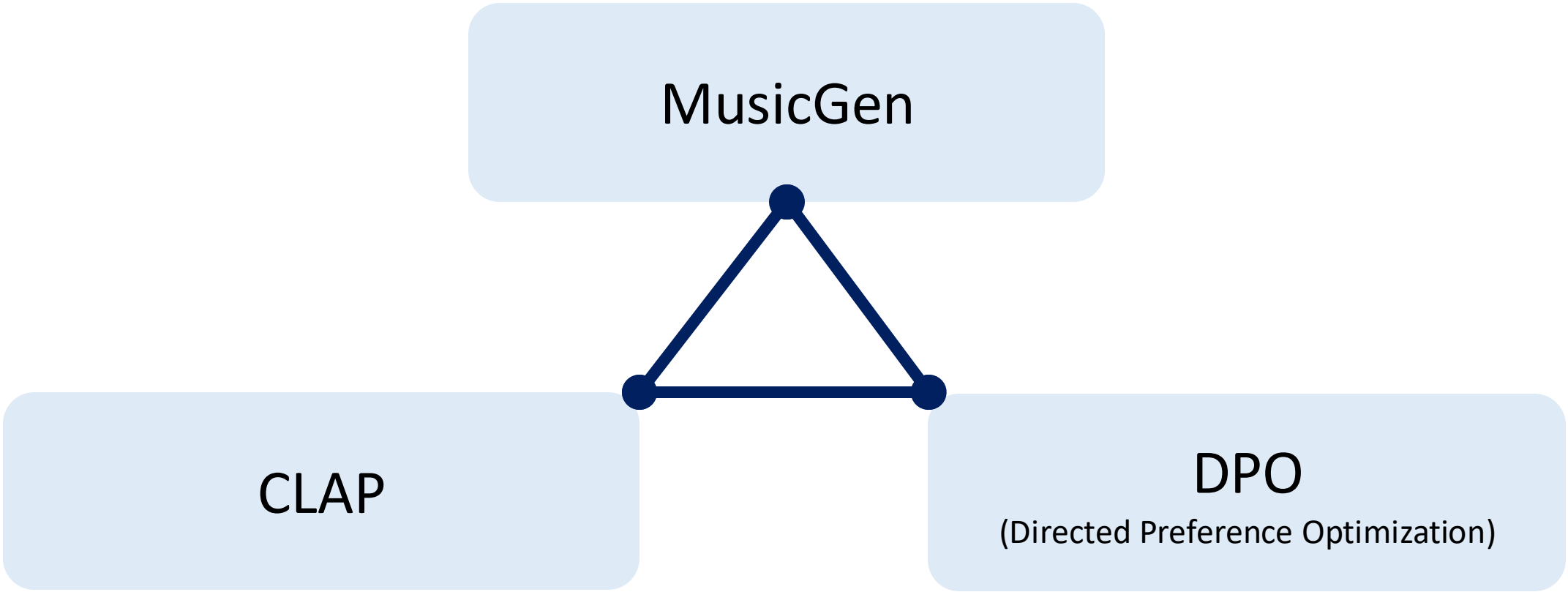
# Methodology

## Overview workflow



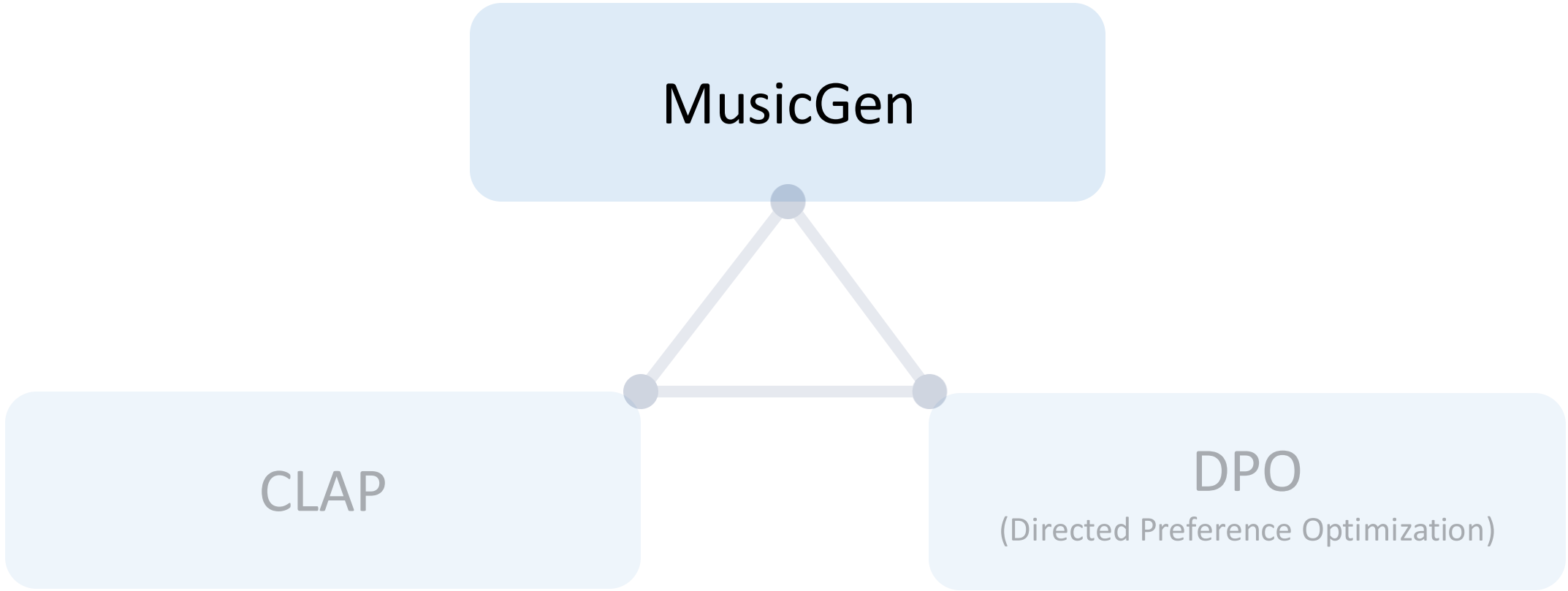
# Methodology

## Modules



# Methodology

## Modules



# Methodology

## MusicGen

- Task
  - text prompt → audio wave
- Model Series
  - Due to memory limitation, we use MusicGen-small (330M params)
  - Pretrained weights are from *facebook/musicgen-small* on Huggingface
- Encoder / Decoder
  - Encodec with 32k sampling rate
  - The generated audio contains 256 tokens per 5 seconds.

# Methodology

## LoRA (Low Rank Adaptation)

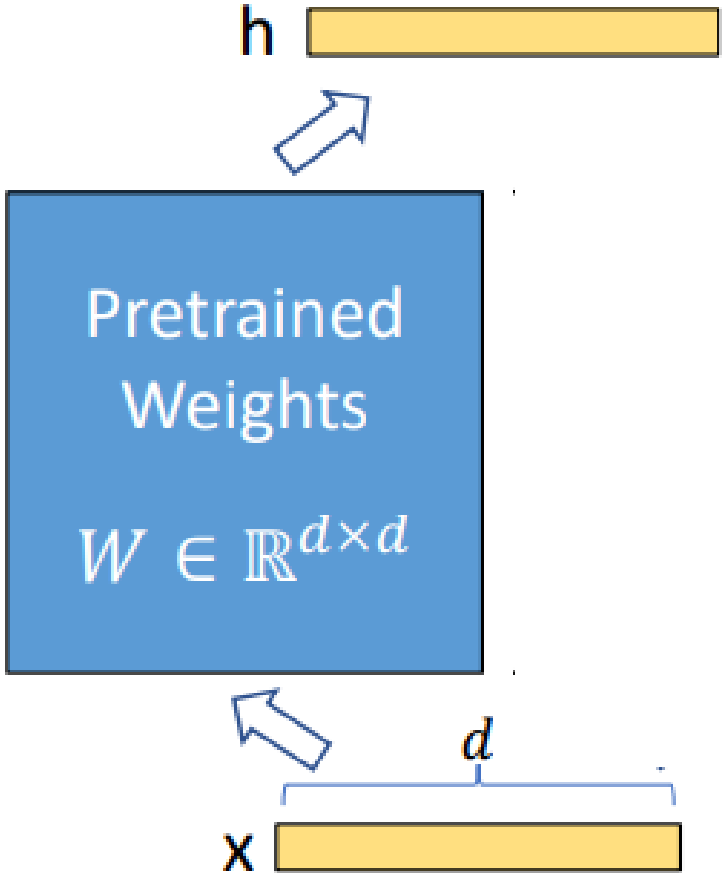
- Why we need LoRA? **LACK OF Computational Resources !!!**
  - MusicGen & CLAP consume up to 20 GB VRAM
  - We want to generate 25s music → about 1280 tokens
- What do LoRA do?
  - Freezing the pretrained weights of the LLM
  - Adding a small number of trainable matrix parameters to the LLM.
  - A and B are low rank matrices, which significantly reduces the number of trainable parameters.

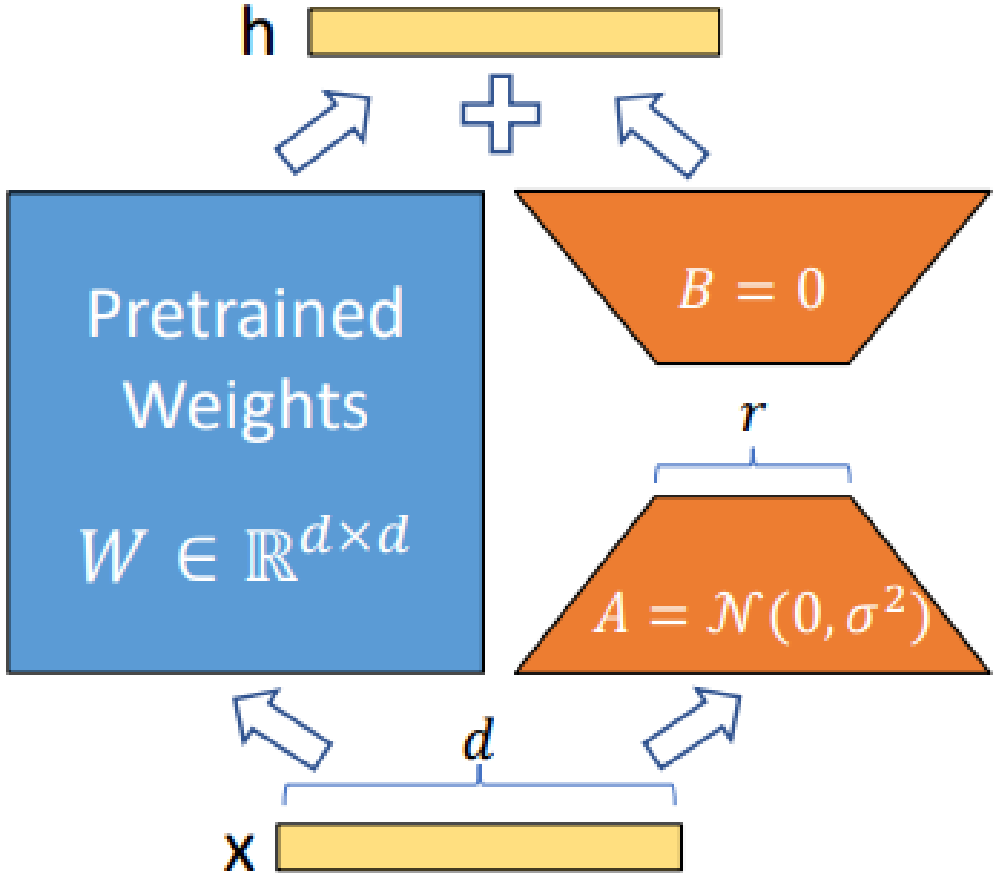
  **→ Reduce the memory usage during fine-tuning !**
- We add LoRA module in Q matrices and V matrices of the transformers.

# Methodology

## LoRA (Low Rank Adaptation)

- Before Fine-Tuning

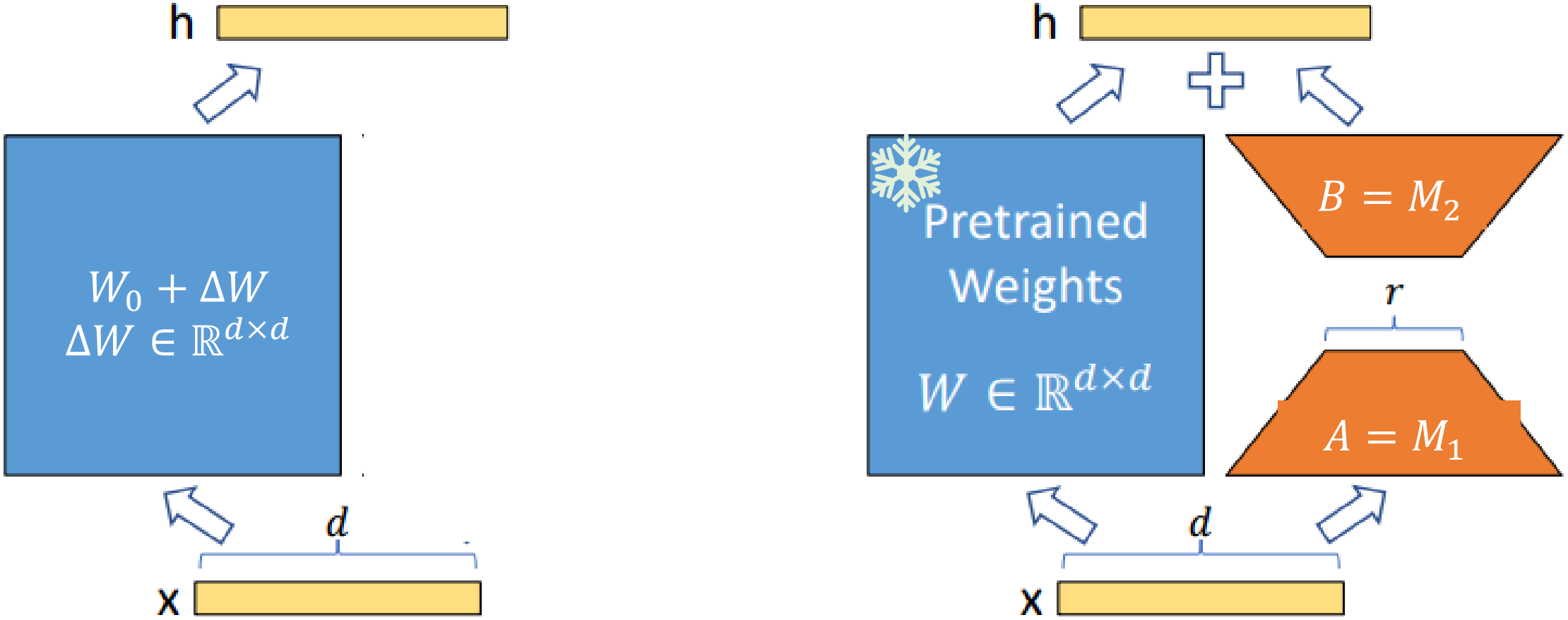


# Methodology

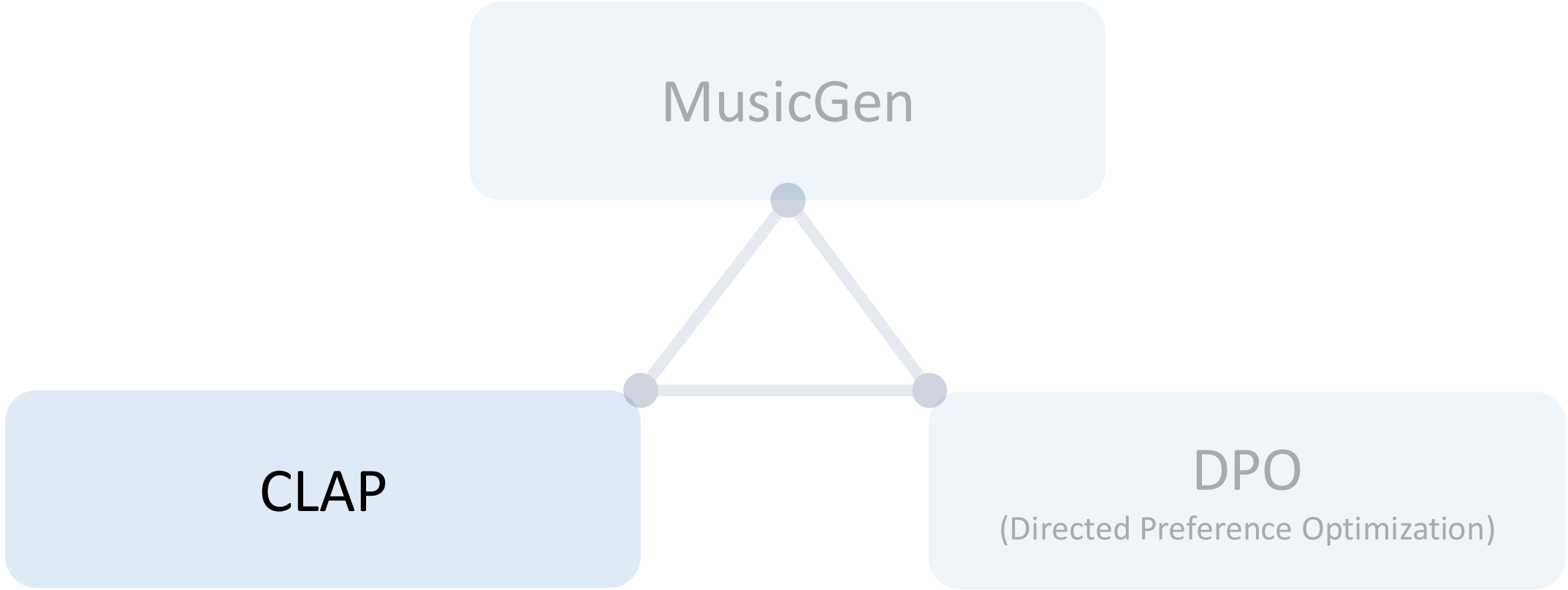
## LoRA (Low Rank Adaptation)

- After Fine-Tuning, $\Delta W$ is $d^2$ params, while $B \cdot A$ is $2dr$ params



# Methodology

## Modules



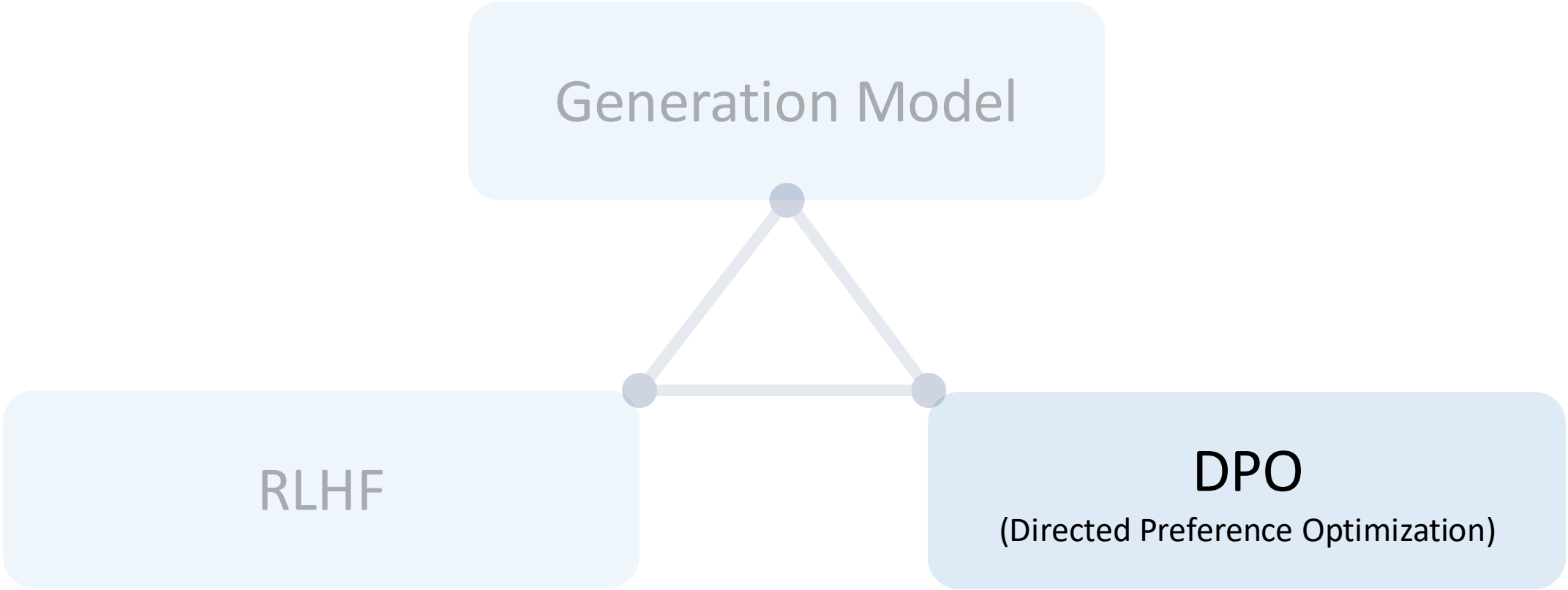
# Methodology

## CLAP

- Why we need CLAP?
  - Enable the MusicGen to understand the relationship between audio and text descriptions
- Pretrained CLAP
  - Trained on longer prompts which are not specified to classical music
- Fine-tuned CLAP
  - *Input text* is the composer's style, composition, movement, and ensemble for each piece of music
  - *Audio chunks* are separated 25s audio chunks from the MusicNet
- CLAP Loss (Loss for fine-tune MusicGen)
  - Computes the similarity ($sim$) between the embeddings of audio and the text prompt
  - $\mathcal{L}_{CLAP} = (1 - sim)$

# Methodology

## Modules



# Methodology

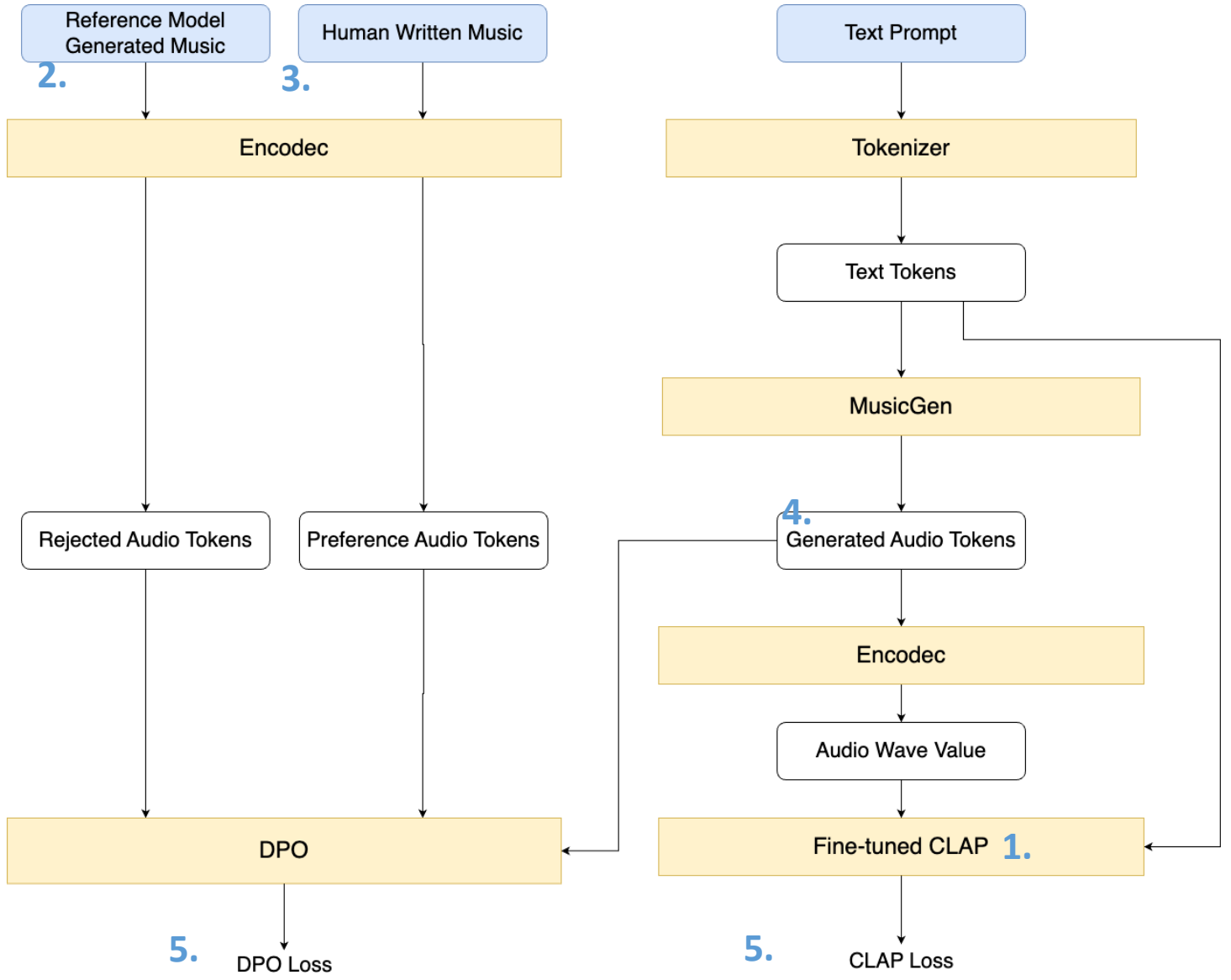
## DPO

- Motivation
  - Most RLHF need to train a reward function

    → Require human to rank the preference
  - DPO Only need a preferable – unpreferable pairing data
- Loss
  - $\mathcal{L}_{DPO}(\pi_\theta, \pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left(\log\sigma\left(\beta\cdot\log\left(\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}\right)-\beta\cdot\log\left(\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right)\right)$
  - Force the model to generate preferable output while push away from unpreferable ones
  - Leverage the reference model as a constraint to prevent from high diversion between the pre-trained and tuned model

# Methodology

## Overview workflow



03

# Experiment Results

# Experiment Results

## Experiment Settings

| | |
|---|---|
| Model | • MusicGen-small<br>• 330M params<br>• Pre-trained LAION-CLAP<br>• LoRA r: 8<br>• LoRA alpha: 8 |
| Training Configuration | • 20 epochs<br>• Learning rate: 0.0005<br>• Optimizer: AdamW |
| Computation Resources | • RTX - 4090<br>• 24GB VRAM |
| Ablation Settings | • DPO Loss + Fine-tuned CLAP Loss<br>• DPO Loss + Pre-trained CLAP Loss<br>• Only DPO Loss |

# Experiment Results

## Evaluation Metrics – KL Divergence

- Purpose
  - Measure distribution distance between the human-written music and our generated music
- Method
  - Calculate the audio embeddings from a state-of-the-art classification model
  - Using the embeddings to derive the KL divergence for the hand written and generated audio

# Experiment Results

## Evaluation Metrics – KL Divergence

- The more fine-tuning loss components, the higher the KL score is.
- Pretrained CLAP has higher KL score comparing to Fine-tuned.
- Though the higher KL divergence indicate higher deviation from the distribution of human-written music, it may provide higher creativity and diversity.

| | Only DPO | DPO & Pretrained CLAP | DPO & Fine-tuned CLAP | Pretrained MusicGen (w/o Fine-tune) |
|---|---|---|---|---|
| KL Div. | $1.1776 \pm 0.59$ | $1.2585 \pm 0.56$ | $1.1952 \pm 0.57$ | $1.0811 \pm 0.61$ |

# Experiment Results

## Evaluation Metrics – CLAP Score

- Purpose
  - Verify the effectiveness of the CLAP module
  - Measuring the alignment between input text and generated music
- Method
  - Using the pre-trained CLAP to calculate the cosine similarity between input prompt and the generated audio.

# Experiment Results

## Evaluation Metrics – CLAP Score

- the CLAP score indeed increased after adding the CLAP loss
- Using Pretrained CLAP would degrade the CLAP score, even lower than the one without CLAP model.
  → We think it is due to the distribution shifting between the pretrained dataset and our classical music dataset.

| | Only DPO | DPO & Pretrained CLAP | DPO & Fine-tuned CLAP | Pretrained MusicGen (w/o Fine-tune) |
|---|---|---|---|---|
| CLAP Score | $0.2025 \pm 0.011$ | $0.2002 \pm 0.011$ | $0.2035 \pm 0.010$ | $0.1988 \pm 0.012$ |

# Experiment Results

## Listening Samples I

**Prompt: Solo Violin in E major with Baroque style**



**w/o Fine-tune**



**Only DPO Loss**



**DPO &**
**Pretrained CLAP**



**DPO &**
**Fine-tuned CLAP**

# Experiment Results

## Listening Samples II

**Prompt: Wind Quintet in F Major with Classical style**



**w/o Fine-tune**



**Only DPO Loss**



**DPO &**
**Pretrained CLAP**



**DPO &**
**Fine-tuned CLAP**

04

# Conclusion

# Conclusion

## Limitations

01 The key and instruments are difficult to control even after the fine-tuning.

02 Though CLAP has been fine-tuned on the classical music dataset, the effectiveness is still limited due to the insufficient data.

03 MusicGen performs better on generating Baroque style music due to the class imbalance in the dataset.

04 Since we using only MusicGen-small, its performance improvement remains limited even after fine-tuning.

# Conclusion

## Key Takeaways

01 With the DPO, MusicGen can generate more human-like music.

02 With CLAP model, MusicGen is able to capture more information of the text prompt such as the style and instruments.

03 Applying fine-tuned CLAP can lead to better evaluation values.

05

# References

# References


[1] Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., ... & Défossez, A. (2024). Simple and controllable music generation. *Advances in Neural Information Processing Systems*, *36*.

[2] Elizalde, B., Deshmukh, S., Al Ismail, M., & Wang, H. (2023, June). Clap learning audio concepts from natural language supervision. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE.

[3] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2024). Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, *36*.

[4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

[5] Kreuk, F., Synnaeve, G., Polyak, A., Singer, U., Défossez, A., Copet, J., ... & Adi, Y. (2022). Audiogen: Textually guided audio generation. *arXiv preprint arXiv:2209.15352*.